1 Assignment: 1

▶ How to add and subtract two n-bit integers?

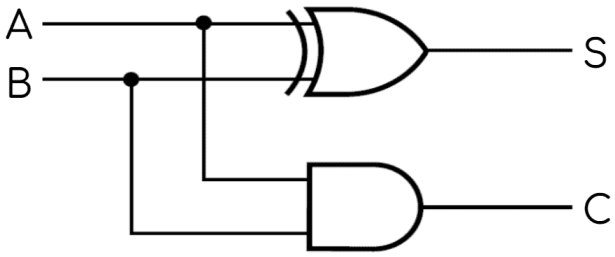
basic addition operations:

$$0+0=0, \quad 1+0=1, \quad 0+1=1, \quad 1+1=\underset{\text{(Carry)}}{\overset{\downarrow}{10}}$$

To simply add two 1-bit numbers we need a **half adder circuit**.

Equations:

$$S = \bar{A}B + A\bar{B} = A \oplus B$$
$$C = A \cdot B$$



Truth Table:

A	B	Sum(S)	Carry out(C_{out})
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

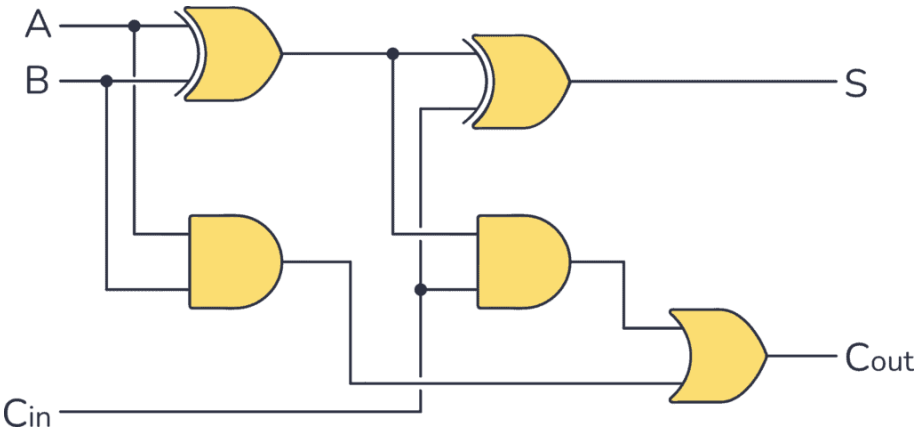
To add two 2-bit numbers we need a **full adder circuit**.

Equations:

$$\begin{aligned} S &= \bar{C}_{\text{in}}\bar{A}B + \bar{C}_{\text{in}}A\bar{B} + C_{\text{in}}\bar{A}\bar{B} + C_{\text{in}}AB \\ &= A \oplus B \oplus C \\ C_{\text{out}} &= \bar{C}_{\text{in}}AB + C_{\text{in}}\bar{A}B + C_{\text{in}}A\bar{B} + C_{\text{in}}AB \\ &= C_{\text{in}}(A \oplus B) + AB \end{aligned}$$

Truth Table:

C_{in}	A	B	Sum(S)	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Add n-bit integers:

We need $(n-1)$ number of full adders and 1 half adder.

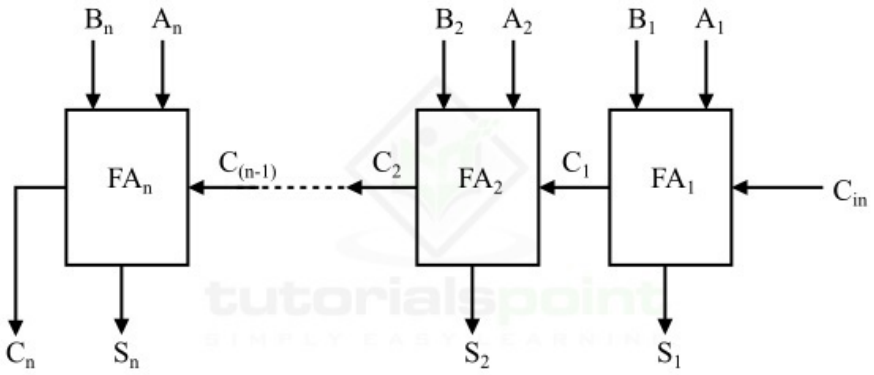


Figure 2 - N-Bit Parallel Adder

▶ Subtraction Rules

$$0-0=0, \quad 0-1=\underset{\substack{\downarrow \\ \text{borrow}}}{11}, \quad 1-0=1, \quad 1-1=0$$

Subtraction of two 1-bit binary numbers:
We use a **Half Subtractor**.

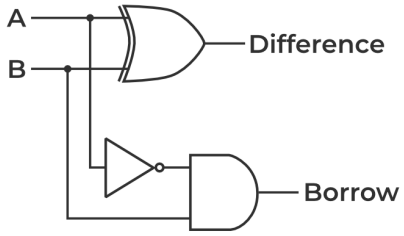
Equations:

$$D = A \oplus B$$
$$B_{\text{out}} = \bar{A}B$$

Truth Table:

A	B	Difference (D)	Borrow (B_{out})
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Circuit Diagram:

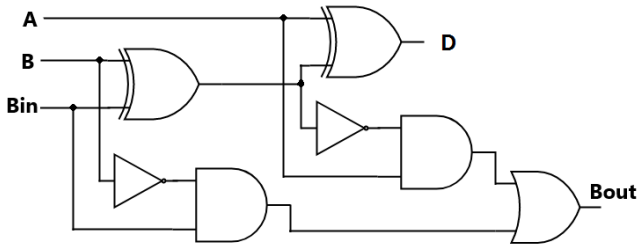


Full Subtractor: Used for subtraction between two 1-bit binary numbers and considers the borrow of the previous bit.

Equations:

$$D = A \oplus B \oplus B_{\text{in}}$$
$$B_{\text{out}} = \bar{A}B + B_{\text{in}}(A \oplus B)'$$

Circuit Diagram:



Truth Table:

A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2's Complement Subtraction

Subtraction of A by B is obtained by taking the 2's complement of B and adding it to A.

- The 2's complement of B is obtained by taking the 1's complement of B and adding 1 to the LSB (Least Significant Bit).

Parallel Adder / Subtractor Circuit

We use n Full Adders. The mode input control line M is connected with the carry input of the LSB (C_{in}). The control line decides whether addition or subtraction is performed.

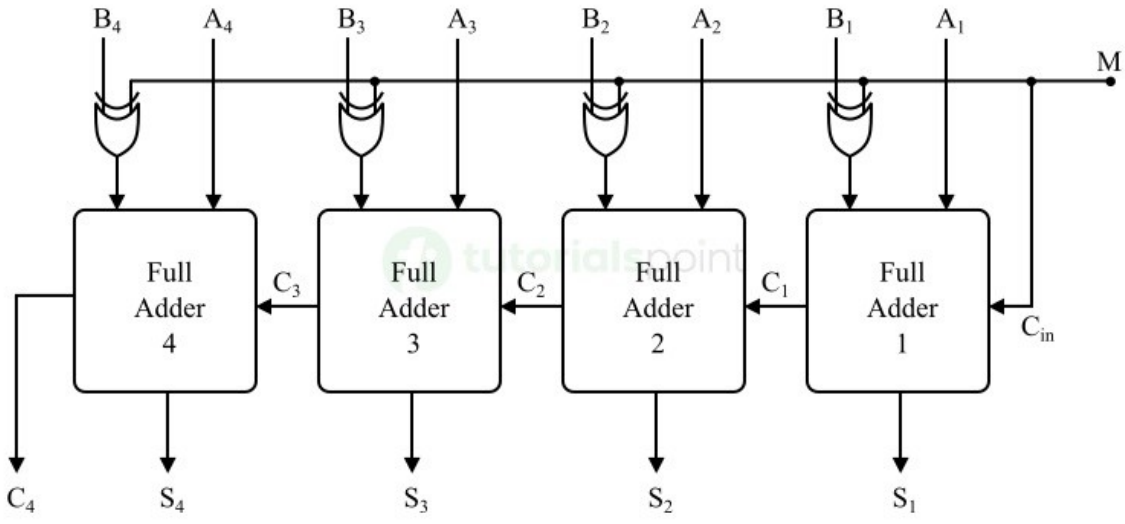


Figure 1: 4-bit Binary Parallel Adder / Subtractor Circuit

Working Operation:

1. When $M = 1$ (Subtractor Mode):

- $B \oplus 1 = \bar{B}$ (produces 1's complement of B).

- The carry input $C_{\text{in}} = 1$ is added to the LSB.
- Hence, complemented B along with 1 added via carry input yields A+(2's complement of B).
- Thus, the **subtraction operation** $(A - B)$ is performed.

2. When $M = 0$ (Adder Mode):

- $B \oplus 0 = B$.
- The carry input $C_{\text{in}} = 0$.
- Hence, the normal **addition operation** $(A + B)$ is performed.

2 Assignment: 2

∗ C code to find GCD of two integers :

Let a, b $(b \neq 0)$ be two integers (+ve integers).

By Euclidean Algorithm, $\exists$ unique integers q and r such that

$$a = bq + r \quad \text{where } 0 \leqslant r < b$$

So, $\gcd(a, b) = \gcd(b, r)$

We repeat the process until after division remainder is zero.

$$\begin{aligned} r_0 &= q_1 r_1 + r_2 \quad (r_2 < r_1) \\ r_1 &= q_2 r_2 + r_3 \quad (r_3 < r_2) \end{aligned}$$

$$\begin{aligned} r_0 &= q_1(q_2 r_2 + r_3) + r_2 \\ &= (q_1 q_2 + 1) r_2 + q_1 r_3 \\ r_0 &> (q_1 q_2 + 1) r_2 > 2 r_2 \\ \implies r_2 &< \frac{r_0}{2} \end{aligned}$$

Remainder is cut in half at least every two steps. So, for n-bit integer, as max value is 2^n, algorithm terminates in **max $2n$ steps**.

C Program to Find GCD of Two Integers

```c
unsigned int find_gcd(unsigned int a, unsigned int b) {
    while (b != 0) {
        unsigned int rem = a % b;
        a = b;
        b = rem;
    }
    return a;
}
```

To calculate the time:

```c
#include <stdio.h>
#include <time.h>

unsigned int find_gcd(unsigned int a, unsigned int b) {
    while (b != 0) {
        unsigned int rem = a % b;
        a = b;
        b = rem;
    }
    return a;
}

int main() {
    clock_t t1, t2;
    int i, repcount;
    float timeTaken;

    unsigned int x,y;
    unsigned int result;
    printf("Enter two positive integers: ");
    scanf("%u %u",&x,&y);
    repcount = 1000000;

    t1 = clock();
    for (i = 0; i < repcount; i++) {
        result = find_gcd(x, y);
    }

    t2 = clock();

    timeTaken = (((float)(t2 - t1) / (float)CLOCKS_PER_SEC) / (float)repcount);

    printf("GCD of %u and %u is: %u\n", x, y, result);
    printf("Time taken = %f seconds\n", timeTaken);

    return 0;
}
```

References

[1] https://www.geeksforgeeks.org/digital-logic/4-bit-binary-adder-subtractor/

[2] Google Gemini : execution time using clock()